

Backtracking e Bruteforce: Uma análise de suas implementações na solução do Tango

Bernardo Alvim¹, Gabriela Alvarenga¹,
Marcos Alberto¹, Mateus Araujo¹, Pedro Seabra¹

¹ Instituto de Ciências Exatas e Informática
Pontifícia Universidade Católica de Minas Gerais (PUC Minas)

Abstract. Introduction: *Constraint Satisfaction Problems (CSPs) constitute a fundamental class of computational problems in which a solution must simultaneously satisfy a set of predefined rules. Logic puzzles, such as Tango, are ideal laboratories for studying search strategies because they require filling a grid with exactly two symbols under five mutually constraining rules.*

Objective: *This work models the Tango puzzle as a CSP and comparatively analyzes two resolution approaches, Brute Force and Backtracking, evaluating how early constraint checking affects the size of the explored search space and practical execution time.* **Methodology:** *Both solvers were implemented in Java with a two-layer architecture that separates a generic CSP engine from the Tango-specific domain logic. Six boards ranging from 4×4 to 16×16 were used as benchmarks, with metrics collected for nodes visited, constraint checks, and pruning events.* **Results:** *Backtracking solved all boards including 16×16 in under 335 ms with a pruning rate of 99.94%. Brute Force proved infeasible beyond 6×6 boards. For the hardest 6×6 instance, Backtracking required 78 constraint checks against 2.8 billion for Brute Force, a ratio of over 36,000,000 $\times$, demonstrating the decisive impact of pruning on search tractability.*

Keywords *Backtracking, Brute Force, Constraint Satisfaction Problems, Tango, Pruning.*

Resumo. Introdução: *Os Problemas de Satisfação de Restrições (CSPs) constituem uma classe fundamental de problemas computacionais em que uma solução deve satisfazer simultaneamente um conjunto de regras pré-definidas. Quebra-cabeças lógicos como o Tango são laboratórios ideais para o estudo de estratégias de busca, pois exigem o preenchimento de uma grade com exatamente dois símbolos sob cinco regras mutuamente restritivas.* **Objetivo:** *Este trabalho modela o Tango como um CSP e analisa comparativamente duas abordagens de resolução, Força Bruta e Backtracking, avaliando como a verificação antecipada de restrições afeta o tamanho do espaço de busca explorado e o tempo de execução prático.* **Metodologia:** *Ambos os solvers foram implementados em Java com uma arquitetura de duas camadas que separa um motor CSP genérico da lógica específica do domínio Tango. Seis tabuleiros variando de 4×4 a 16×16 foram utilizados como benchmarks, com coleta de métricas de nós visitados, verificações de restrição e eventos de poda.* **Resultados:** *O Backtracking solucionou todos os tabuleiros, incluindo o de 16×16 , em menos de 335 ms com taxa de poda de 99,94%. A Força Bruta*

mostrou-se inviável além de tabuleiros 6×6 . Para a instância 6×6 mais difícil, o Backtracking exigiu 78 verificações de restrição contra 2,8 bilhões da Força Bruta, uma razão superior a $36.000.000\times$, demonstrando o impacto decisivo da poda sobre a tratabilidade da busca.

Palavras-Chave Backtracking, Força Bruta, Problemas de Satisfação de Restrições, Tango, Poda.

1. Introdução

Em Ciência da Computação, os Problemas de Satisfação de Restrições (CSPs) descrevem situações em que o objetivo é encontrar um estado que satisfaça simultaneamente um conjunto de regras ou limitações pré-definidas. A resolução ingênua dessas instâncias por enumeração completa, denominada Força Bruta, testa todas as combinações possíveis sem nenhum critério de eliminação antecipada, o que resulta em crescimento exponencial do tempo conforme o tamanho do problema aumenta. A técnica de Backtracking surge como um refinamento: a solução é construída de forma incremental e as restrições são avaliadas a cada passo, podando subárvores inteiras do espaço de busca assim que uma violação é detectada [Cormen et al. 2012].

Jogos lógicos são ambientes naturais para o estudo dessas técnicas, pois possuem regras formalizáveis e soluções verificáveis. O Tango, jogado em uma grade $N \times N$ onde cada célula deve receber um de dois símbolos, Sol ou Lua, é um representante direto dessa categoria: suas cinco regras impõem restrições locais de adjacência, equilíbrio global por linha e coluna, e condições binárias entre pares de células específicos. Todo tabuleiro Tango bem formulado possui exatamente uma solução, o que garante que qualquer algoritmo correto de busca a encontrará.

Este trabalho apresenta a modelagem do Tango como CSP, a implementação de Força Bruta e Backtracking em Java com uma arquitetura de duas camadas que separa o motor de busca genérico da lógica do domínio, e a análise comparativa de desempenho sobre seis tabuleiros de tamanhos distintos. O código-fonte completo está disponível em github.com/marcosffp/tp2-fpaa-tango-solver.

2. Fundamentação Teórica

Esta seção apresenta os conceitos que embasam a análise desenvolvida nas seções seguintes: a definição formal de CSPs, o mecanismo da Força Bruta e os princípios do Backtracking com poda.

2.1. Problemas de Satisfação de Restrições

Um CSP é formalmente definido por uma tripla (X, D, C) , onde $X = \{x_1, \dots, x_n\}$ é o conjunto de variáveis, $D = \{D_1, \dots, D_n\}$ é o conjunto de domínios, um para cada variável, e C é o conjunto de restrições que definem quais atribuições são permitidas [Cormen et al. 2012]. No Tango, as variáveis são as células da grade, o domínio de cada célula é $\{\text{Sol}, \text{Lua}\}$, e as restrições codificam as cinco regras do jogo. Uma solução é uma atribuição completa de valores às variáveis que satisfaz todas as restrições em C simultaneamente.

2.2. Força Bruta

A Força Bruta constrói uma árvore binária de decisão sobre as k células vazias do tabuleiro: em cada nível, escolhe-se uma célula e ramifica-se para Sol ou para Lua. As restrições são verificadas apenas nas folhas da árvore, ou seja, somente após o preenchimento completo do tabuleiro. O número de folhas é 2^k , e o custo de avaliar cada folha é proporcional a N^2 , tornando a complexidade total $O(2^k \cdot N^2)$ no pior caso. A vantagem da abordagem é a simplicidade de implementação; a desvantagem é que nenhuma combinação parcial inviável é descartada durante a geração.

2.3. Backtracking

O Backtracking constrói a solução de forma incremental, atribuindo um valor a uma variável por vez e verificando as restrições imediatamente após cada atribuição, mesmo que o tabuleiro ainda esteja parcialmente preenchido. Se o estado parcial já viola alguma restrição, a ramificação inteira abaixo daquele estado é descartada sem ser explorada, o que constitui a operação de poda. O algoritmo recua ao ponto de decisão anterior e tenta o próximo valor disponível, repetindo o processo até encontrar uma solução ou esgotar todas as combinações [Cormen et al. 2012]. A efetividade do Backtracking depende diretamente da capacidade das restrições de detectar inconsistências em estados parciais: quanto mais cedo uma violação é identificada, maior a subárvore eliminada. Se uma poda ocorre com j células ainda por preencher, são eliminados 2^j estados de uma só vez.

3. Modelagem do Problema

3.1. O Tabuleiro e as Cinco Regras

O Tango é jogado em uma grade $N \times N$, onde N é par. Cada célula deve receber obrigatoriamente um dos dois símbolos disponíveis: Sol ou Lua. As cinco regras que definem uma configuração como solução válida são:

- (1) **Preenchimento Completo:** toda célula contém um símbolo ao final.
- (2) **Limite de Adjacência:** não existem três ou mais símbolos idênticos consecutivos, seja na horizontal ou na vertical.
- (3) **Equilíbrio:** cada linha e cada coluna contém exatamente $N/2$ Sóis e $N/2$ Luas.
- (4) **Restrição de Igualdade (=):** células adjacentes conectadas pelo sinal = devem conter o mesmo símbolo.
- (5) **Restrição de Oposição (×):** células adjacentes conectadas pelo sinal × devem conter símbolos opostos.

3.2. Representação em Estruturas de Dados

O tabuleiro é representado pela classe `Board`, que armazena internamente um array unidimensional de objetos `Cell` com N^2 elementos. A escolha por array unidimensional em vez de uma matriz bidimensional decorre do fato de que os algoritmos de busca referenciam células por um único inteiro, o índice linear calculado pela fórmula $i = \text{linha} \times N + \text{coluna}$, o que permite acesso em $O(1)$ sem conversão adicional. O método `getCell(row, col)` realiza essa conversão quando o acesso por coordenadas é necessário, como nas regras que percorrem linhas e colunas.

Cada `Cell` armazena o símbolo atual, representado pelo enum `Symbol` com valores `SUN` e `MOON`, e um campo boolean `fixed` que indica se a célula é uma dica

fixada pelo puzzle. Células fixas têm seu símbolo definido pelo parser de entrada e nunca são modificadas durante a busca. Quando `symbol` é `null`, a célula está vazia. O método `setSymbol()` protege essa invariante lançando uma exceção ao detectar tentativa de modificação de célula fixa.

As restrições de igualdade e oposição entre pares de células são modeladas pela classe `Constraint`, que armazena os índices lineares das duas células envolvidas e o tipo da relação por meio do enum `ConstraintType`, com valores `EQUAL` e `OPPOSITE`. A lista de restrições é criada como imutável no construtor do `Board` com `List.copyOf()`, garantindo que a definição do puzzle não seja alterada durante a resolução. Por essa mesma razão, quando o algoritmo precisa de uma cópia independente do tabuleiro, o método `BoardFactory.deepCopy()` copia apenas os objetos `Cell`, que são mutáveis, e reutiliza a mesma lista de restrições sem copiá-la.

Os tabuleiros são lidos de arquivos `.txt` pelo `BoardParser`: a primeira linha contém N , as N linhas seguintes descrevem cada linha da grade com tokens `S` (Sol fixo), `M` (Lua fixa) ou `.` (célula vazia), e as linhas restantes descrevem as restrições no formato `= 11 c1 12 c2` ou `x 11 c1 12 c2`, onde as coordenadas são baseadas em zero.

3.3. Arquitetura de Duas Camadas

O projeto é organizado em duas camadas independentes que se comunicam exclusivamente por interfaces Java. A camada `csp/` contém os motores de busca genéricos e os contratos abstratos que eles precisam: `State<S>`, com os métodos `assign`, `unassign`, `isComplete` e `clone`; `ConstraintChecker<S>`, com `isValid(S)`; `VariableSelector<S>`, com `selectVariable(S)`; e `ValueIterator<S>`, com `getValues(S, int)`. Essa camada não importa nada relacionado ao Tango: não conhece Sol, Lua, tabuleiro, linha ou coluna. A camada `tango/` contém toda a lógica específica do domínio, implementando os contratos da camada CSP e traduzindo operações genéricas para operações concretas no tabuleiro. A montagem dos componentes ocorre no `Main.java`:

```
TangoConstraintChecker checker = new TangoConstraintChecker(List.of(
    new ConsecutiveRule(), new BalanceRule(),
    new EqualRule(),      new OppositionRule()
));
TangoVariableSelector selector = new TangoVariableSelector();
TangoValueIterator iterator = new TangoValueIterator();

new BacktrackingEngine<>(checker, selector, iterator).solve(adapter);
```

O `TangoBoardAdapter` implementa `State<TangoBoardAdapter>` e traduz as operações genéricas do motor para operações concretas no `Board`: `assign(i, v)` converte o índice i em uma célula e o valor v no símbolo correspondente por `Symbol.values()[v]`, enquanto `unassign(i)` chama `setSymbol(null)` sobre a mesma célula. O `TangoVariableSelector` seleciona sempre a primeira célula vazia em ordem linear crescente. O `TangoValueIterator` retorna `[0, 1]` para todas as células, onde 0 representa Sol e 1 representa Lua, de forma consistente com a convenção adotada no adaptador.

4. Estratégias de Resolução

As duas estratégias implementadas compartilham a mesma estrutura recursiva e os mesmos componentes de seleção de variável e iteração de valores, diferindo exclusivamente no momento em que as restrições são avaliadas. Na Força Bruta, a verificação ocorre somente ao término do preenchimento; no Backtracking, ocorre após cada atribuição individual. Essa diferença, aparentemente pontual, é o que determina a eficiência de cada abordagem e, em instâncias maiores, a própria viabilidade da execução.

4.1. Força Bruta

A Força Bruta enumera recursivamente todas as combinações de símbolos para as células vazias e verifica as cinco regras apenas quando o tabuleiro está completamente preenchido. O método central, `generate()`, é apresentado a seguir:

```
private S generate(S state) {
    nodesVisited++;

    if (state.isComplete()) {
        attempts++;
        return checker.isValid(state) ? state : null;
    }

    OptionalInt variableOpt = selector.selectVariable(state);
    if (variableOpt.isEmpty()) return null;
    int variable = variableOpt.getAsInt();

    for (int value : iterator.getValues(state, variable)) {
        state.assign(variable, value);
        S candidate = generate(state);
        if (candidate != null) return candidate;
        backtracks++;
        state.unassign(variable);
    }
    return null;
}
```

Cada chamada a `generate()` representa um nó na árvore de busca, e o contador `nodesVisited` é incrementado a cada entrada. Quando `state.isComplete()` é verdadeiro, o tabuleiro foi completamente preenchido: `checker.isValid(state)` aplica as cinco regras sobre o estado final e retorna o estado caso todas sejam satisfeitas, ou `null` caso contrário. O ponto central da abordagem é que `checker.isValid(state)` não é chamado em nenhum outro ponto do método; toda verificação ocorre exclusivamente em tabuleiros completos. Quando um caminho não leva a solução, `state.unassign(variable)` desfaz a atribuição e o próximo valor é tentado. Se ambos os valores de uma célula forem esgotados sem sucesso, o método retorna `null`, propagando o retrocesso para o nível anterior.

4.2. Backtracking

O Backtracking aplica as restrições imediatamente após cada atribuição individual, antes de avançar para a próxima célula. O método central, `backtrack()`, é apresentado a seguir:

```
private void backtrack(S state) {
    nodesVisited++;

    if (state.isComplete()) {
        if (firstSolution == null) firstSolution = state.clone();
        return;
    }

    OptionalInt variableOpt = selector.selectVariable(state);
    if (variableOpt.isEmpty()) return;
    int variable = variableOpt.getAsInt();

    for (int value : iterator.getValues(state, variable)) {
        state.assign(variable, value);
        attempts++;

        if (checker.isValid(state)) {
            backtrack(state);
            if (firstSolution != null) return;
        } else {
            backtracks++;
        }

        state.unassign(variable);
    }
}
```

A condição de parada da recursão é `state.isComplete()`: quando todas as células estão preenchidas e nenhuma regra foi violada ao longo do caminho, o estado é clonado e armazenado em `firstSolution`. O clone é necessário porque `state` é o objeto mutável de trabalho compartilhado por todos os níveis da recursão: as chamadas ainda pendentes de retorno continuarão desfazendo atribuições nele, e salvar a referência direta corromperia a solução já encontrada.

Quando `checker.isValid(state)` retorna false após uma atribuição, o estado parcial já viola alguma regra: `backtracks++` registra a poda, e o laço avança para o próximo valor sem recorrer. Toda a subárvore de combinações derivadas daquele estado parcial é eliminada sem ser gerada. Quando a chamada recursiva retorna com `firstSolution != null`, o algoritmo encerra imediatamente os níveis superiores sem tentar o segundo valor da célula atual.

CrITÉRIOS de Poda: as Cinco Regras sobre Estados Parciais

As quatro classes de regra, `ConsecutiveRule`, `BalanceRule`, `EqualRule` e `OppositionRule`, implementam a interface `Rule` e são invocadas pelo `TangoConstraintChecker` por meio de `stream().allMatch()`, que

interrompe a avaliação na primeira regra que falhar. Todas as regras tratam células nulas, ainda não preenchidas, como inconclusivas: a ausência de símbolo nunca gera uma violação.

A `ConsecutiveRule` percorre cada linha e coluna mantendo um contador de símbolos idênticos consecutivos. Ao encontrar uma célula vazia, o contador é zerado, pois não se pode afirmar que a sequência continua por aquela posição. Se o contador ultrapassa dois antes do fim da linha, a regra retorna `false` imediatamente.

A `BalanceRule` aplica uma verificação antecipada mais sofisticada. Além de detectar quando um símbolo já ultrapassou o limite $N/2$, ela verifica se ainda é matematicamente possível atingir o equilíbrio nas posições restantes, utilizando o campo `remaining`, que conta as células ainda vazias na linha ou coluna:

```
if (sunCount > half || moonCount > half) return false;
if (sunCount + remaining < half)         return false;
if (moonCount + remaining < half)        return false;
```

A segunda e a terceira verificações detectam quando, mesmo preenchendo todas as células vazias com um único símbolo, o equilíbrio não seria atingido. Essa propriedade transforma a `BalanceRule` em um critério de poda que rejeita estados parciais que nunca poderiam levar a uma linha ou coluna balanceada, independentemente dos valores atribuídos às células restantes.

A `EqualRule` e a `OppositionRule` percorrem a lista de restrições do tabuleiro e avaliam cada par de células apenas quando ambas já foram preenchidas. Caso uma delas ainda seja nula, a restrição é ignorada naquele momento. Quando ambas estão preenchidas, a `EqualRule` verifica se os símbolos são iguais e a `OppositionRule` verifica se são opostos, retornando `false` em caso de violação.

5. Exemplos de Execução

Os experimentos foram executados em uma máquina com as seguintes especificações:

- **Processador:** Apple M5 (10 núcleos: 4 de desempenho + 6 de eficiência)
- **Memória RAM:** 16 GB LPDDR5 (Unified Memory)
- **Sistema operacional:** macOS 15.5 (Darwin 25.5.0)
- **JRE:** OpenJDK 21.0.10, modo Mixed Server JIT, sem flags de tuning

O trecho abaixo apresenta a saída simplificada do terminal para o tabuleiro 4×4 , que é o único caso em que ambos os algoritmos executam em tempo diretamente comparável. O programa usa S para Sol, L para Lua e exibe os sinais de restrição = e $\times$ posicionados entre as células na grade renderizada.

```
=== Backtracking =====

Tabuleiro 4x4 -- grade inicial:
S . . .
. . . .
. . S .
. . . S

Solucao encontrada:
S S L L
L L S S
L S S L
S L L S

Tempo                : 1,90 ms
Nos visitados        : 15
Verificacoes de restricao : 23
Podas (restricao violada) : 9

=== Forca Bruta =====

Solucao encontrada (identica ao Backtracking):
Tempo                : 3,92 ms
Nos visitados        : 7.772
Verificacoes de restricao : 3.884
Podas (restricao violada) : 0
```

O programa detecta automaticamente tabuleiros maiores que 6×6 e omite a execução da Força Bruta, exibindo uma mensagem de aviso com o tamanho do espaço (2^{49} para o 8×8 e 2^{224} para o 16×16) para justificar a decisão. A Tabela 1 consolida os resultados para todos os tabuleiros testados.

A Tabela 2 resume o comparativo direto entre os dois algoritmos para os quatro tabuleiros em que ambos executaram.

Para o 6×6 difícil, a diferença de 36 milhões de vezes no número de verificações não é compensada por nenhum fator de custo unitário por verificação: a Força Bruta verifica tabuleiros completos, enquanto o Backtracking verifica estados parciais que são mais baratos de avaliar. A diferença real de comportamento é estrutural: as 32 podas do Backtracking eliminaram subárvores inteiras que a Força Bruta percorreu célula a célula.

Tabela 1. Desempenho por execução — Backtracking (BT) e Força Bruta (FB)

Tabuleiro	Alg.	Tempo	Nós	Verif.	Podas
4 × 4	BT	1,90 ms	15	23	9
4 × 4	FB	3,92 ms	7.772	3.884	0
6 × 6 fácil	BT	146 µs	32	48	17
6 × 6 fácil	FB	9,76 s	514.505.331	257.252.659	0
6 × 6 médio	BT	204 µs	37	58	22
6 × 6 médio	FB	16,83 s	785.176.951	392.588.469	0
6 × 6 difícil	BT	112 µs	47	78	32
6 × 6 difícil	FB	122,40 s	5.686.273.731	2.843.136.859	0
8 × 8	BT	383 µs	183	338	156
8 × 8	FB	inviável	—	—	—
16 × 16	BT	335,81 ms	206.771	413.428	206.658
16 × 16	FB	inviável	—	—	—

Tabela 2. Comparativo Backtracking vs Força Bruta

Tabuleiro	Speedup BT	Verif. BT	Verif. FB	Razão FB/BT
4 × 4	2×	23	3.884	169×
6 × 6 fácil	66.454×	48	257.252.659	5.359.430×
6 × 6 médio	82.479×	58	392.588.469	6.768.767×
6 × 6 difícil	1.088.373×	78	2.843.136.859	36.450.473×

6. Análise de Complexidade

O espaço de busca do Tango é determinado pelo número de células vazias k no tabuleiro inicial. Com dois símbolos possíveis por célula, existem 2^k atribuições completas distintas, e a Força Bruta precisa, no pior caso, avaliar todas elas. O custo de cada avaliação completa é $O(N^2)$, pois as cinco regras percorrem as N linhas, as N colunas e a lista de restrições do tabuleiro. O custo total da Força Bruta é portanto $O(2^k \cdot N^2)$. A Tabela 3 apresenta o espaço de busca teórico e a viabilidade prática de cada abordagem para os tamanhos testados.

Tabela 3. Espaço de busca e viabilidade por tamanho de tabuleiro

Tamanho	Células vazias (k)	Espaço (2^k)	FB viável?
4 × 4	13	≈ 8.000	Sim (milissegundos)
6 × 6	30–32	$\approx 10^9$ a 4×10^9	Marginalmente (s a min)
8 × 8	49	$\approx 5,6 \times 10^{14}$	Não (estimativa: anos)
16 × 16	224	$\gg 10^{67}$	Impossível

O Backtracking tem complexidade de pior caso também exponencial, mas o fator de poda reduz drasticamente o número de nós explorados na prática. Para o tabuleiro 6 × 6 difícil, o Backtracking visitou 47 nós contra 5.686.273.731 da Força Bruta, uma redução de aproximadamente 100%. Para o 16 × 16, a taxa de poda foi de 99,94%: de 413.429 verificações realizadas, 206.658 resultaram em poda imediata, com o algoritmo encontrando a solução em 335 ms.

Esse comportamento cresce com o tamanho do tabuleiro porque as regras se tornam proporcionalmente mais restritivas em grades maiores: cada atribuição interage com mais células vizinhas já preenchidas e o equilíbrio de cada linha e coluna aproxima-se mais rapidamente do limite $N/2$, aumentando a probabilidade de detectar violações precocemente. Ao contrário do que ocorre com a Força Bruta, cujo custo cresce com o tamanho do espaço de busca, o Backtracking beneficia-se do aumento do tabuleiro porque cada poda elimina subárvores cada vez maiores em proporção ao espaço total.

A propriedade de unicidade da solução do Tango contribui adicionalmente para o desempenho do Backtracking: como existe exatamente uma solução válida, o algoritmo encerra imediatamente ao encontrá-la, sem precisar explorar o espaço restante.

7. Conclusão

Este trabalho modelou o quebra-cabeça Tango como um CSP e implementou dois algoritmos de busca, Força Bruta e Backtracking, avaliando seu desempenho sobre seis tabuleiros de tamanhos distintos. Os resultados obtidos ilustram de forma concreta a diferença entre os dois paradigmas e confirmam o papel decisivo da poda antecipada na tratabilidade de problemas combinatórios.

A Força Bruta, apesar de correta, revelou-se inviável a partir de tabuleiros 8×8 : com 49 células vazias, o espaço de 2^{49} combinações tornaria a execução impraticável. Mesmo para os tabuleiros 6×6 , o algoritmo precisou avaliar até 2,84 bilhões de configurações completas para a instância mais difícil, levando 122 segundos, enquanto o Backtracking encontrou a mesma solução em 112 microssegundos, com apenas 78 verificações de restrição.

O Backtracking escalou sem dificuldade até o tabuleiro 16×16 , resolvido em 335 milissegundos com taxa de poda de 99,94%. Esse resultado evidencia que as cinco regras do Tango são altamente restritivas mesmo em estados parciais: a maioria das atribuições viola ao menos uma regra antes que o tabuleiro esteja completo, e as podas resultantes eliminam subárvores de tamanho exponencial. A separação entre o motor CSP genérico e a lógica específica do Tango possibilitou a substituição do algoritmo de busca, de Backtracking para Força Bruta e vice-versa, sem modificar nenhuma das quatro regras de validação.

A lição central deste trabalho é que o momento em que as restrições são avaliadas determina não apenas a velocidade da solução, mas se o problema é computacionalmente tratável ou não. Em CSPs com regras restritivas como o Tango, a avaliação a cada passo de atribuição transforma um problema exponencialmente intratável em um solucionável em frações de segundo, confirmando empiricamente o argumento teórico de que a efetividade da poda é o fator dominante na busca com retrocesso.

Referências

Cormen, T. H., Leiserson, C. E., Rivest, R. L., e Stein, C. (2012). *Algoritmos: Teoria e Prática*. GEN LTC, Rio de Janeiro, 3 edition.